PROJET À RENDRE

Mini Plateforme de Gestion de Tâches (Mini Task Manager API)

Contexte

Vous devez développer une API Spring Boot de gestion de tâches, la versionner avec Git, modulariser une partie métier publiée dans Nexus, containeriser l'application avec Docker et enfin la déployer automatiquement sur une VM via Ansible (avec installation automatique de Docker).

L'objectif est de mettre en pratique l'ensemble des notions vues en cours : Git, Maven, Nexus, Docker et Ansible.

Nota Bene : Ce projet est à réaliser en groupe de 2 étudiants maximum. Exceptionnellement, tout étudiant n'ayant pas de binôme est autorisé à le réaliser individuellement.

Le délai est fixé à **une semaine** à compter d'aujourd'hui.

Architecture attendue

Vous devrez produire :

1. Un module métier réutilisable publié dans Nexus
2. Une API Spring Boot principale qui consomme ce module
3. Une image Docker de l'application
4. Un déploiement automatisé avec Ansible sur une VM Ubuntu

Partie Maven + Nexus (Modularisation)

Exemple de configuration Maven

Lors de la création du projet, utilisez une nomenclature cohérente.

Pour `task-core`

```
<groupId>sn.isi.l3gl.core</groupId>
<artifactId>task-core</artifactId>
<version>0.0.1-SNAPSHOT</version>
```

Pour `task-api`

```
<groupId>sn.isi.l3gl.api</groupId>
<artifactId>task-api</artifactId>
<version>0.0.1-SNAPSHOT</version>
```

Module 1 : `task-core`

Créer un projet Maven Spring Boot nommé :

```
task-core
```

Le module `task-core` contient :

- Les entités
- La logique métier
- La couche de persistance (JPA Repository)
- La configuration de la connexion MySQL

Il est responsable de l'accès à la base de données MySQL.

Classe Task

Elle doit contenir :

- id
- title
- description
- status (TODO, IN_PROGRESS, DONE)

Classe TaskService

Elle doit contenir les méthodes suivantes :

- createTask()
- listTasks()
- updateStatus()
- countCompletedTasks()

La classe TaskService contient la logique métier :

- Création d'une nouvelle tâche
- Mise à jour du statut d'une tâche
- Liste des tâches avec leurs statuts
- Calcul du nombre de tâches terminées (avec status DONE)

Les tâches doivent être enregistrées dans une base de données MySQL.

Versioning obligatoire

Chaque fonctionnalité doit correspondre à une version :

```
0.0.1-SNAPSHOT → contient seulement createTask()
0.1.0-SNAPSHOT → ajoute listTasks()
0.2.0-SNAPSHOT → ajoute updateStatus()
0.3.0-SNAPSHOT → ajoute countCompletedTasks()
```

Après validation complète :

```
0.3.0 → version RELEASE finale et push sur nexus
```

Vous devez :

- Publier les quatre versions SNAPSHOT dans Nexus local
- Publier la version finale 0.3.0 (release)
- Vérifier que les artefacts sont visibles dans le repository Maven

La version utilisée dans `task-api` doit être la version release `0.3.0` (sans SNAPSHOT).

Partie Spring Boot (Application principale)

Module 2 : `task-api`

Créer un projet Spring Boot nommé :

```
task-api
```

Ce projet doit :

- Ajouter `task-core` version `0.3.0` comme dépendance depuis Nexus
- Utiliser **uniquement** les méthodes exposées par `task-core` dans chaque endpoint
- Aucune logique métier ne doit être créée dans `task-api` ; créer uniquement un contrôleur contenant les endpoints
- Exposer les endpoints REST suivants :

Méthode	Endpoint	Méthode `task-core` utilisée	Description
POST	/api/tasks	`createTask()`	Créer une tâche
GET	/api/tasks	`listTasks()`	Lister toutes les tâches
PUT	/api/tasks/{id}/status	`updateStatus()`	Modifier le statut d'une tâche
GET	/api/tasks/done/count	`countCompletedTasks()`	Nombre de tâches dont le statut est `DONE`

L'application doit fonctionner sur le port :

```
8085
```

Partie Git

Vous devez :

1. Créer deux repositories GitHub :

    - task-core
    - task-api
2. Respecter les bonnes pratiques Git :

- Une branche par fonctionnalité (exemple : feature/create-task)
- Un commit par fonctionnalité
- Messages de commit clairs et significatifs
- Merge de chaque feature dans la branche develop

Minimum attendu :

- 4 commits fonctionnels pour `task-core` (1 par fonctionnalité)
- 1 commit fonctionnel pour `task-api` (contenant toutes les apis ou endpoint REST)

Partie Docker

Dans le projet task-api :

1. Créer un Dockerfile basé sur OpenJDK 17
2. Builder l'image
3. Tester le conteneur en local
4. Push l'image sur Docker Hub

Nom de l'image attendu :

```
dockerhub_votre_username/task-api:latest
```

L'application dans le conteneur doit exposer le port :

```
8085
```

Partie Ansible

Contexte

- Machine 1 : WSL → machine de contrôle
- Machine 2 : VM Ubuntu Server → machine cible

Objectif du Playbook

Créer un playbook Ansible qui automatise complètement le déploiement.

Étape 1 : Préparation de la VM

Le playbook doit :

- Mettre à jour les paquets
- Vérifier si Docker est déjà installé
- Installer Docker uniquement s'il n'est pas présent
- Installer MySQL
- Démarrer et activer Docker
- Démarrer et activer MySQL

Étape 2 : Déploiement de l'application

Le playbook doit :

- Pull l'image depuis Docker Hub
- Stopper le conteneur existant s'il existe
- Supprimer l'ancien conteneur si nécessaire
- Lancer un nouveau conteneur avec :
  - Port 8085 exposé
  - Redémarrage automatique

Étape 3 : Vérification automatique

Le playbook doit inclure une tâche qui vérifie que l'application répond correctement :

```
http://localhost:8085/api/tasks
ou
http://<adresse-ip-vm>:8085/api/tasks
```

- Cette vérification peut se faire avec :
  - Le module `uri` d'Ansible, **ou**
  - Une commande `shell` dans le playbook
- **Test manuel** :

  Vérifiez également dans un navigateur que l'URL `http://<adresse-ip-vm>:8085/api/tasks` fonctionne correctement.

Livrables attendus

Chaque groupe devra remettre :

1. **Liens GitHub publics** des projets `task-core` et `task-api`.
2. **Une vidéo de démonstration** montrant :
   - Le versioning dans Nexus
   - Le build et le push Docker
   - L'exécution du playbook Ansible
   - L'application fonctionnelle sur la VM
3. **Un lien Notion ou un rapport PDF détaillé** contenant :
   - Explication du travail réalisé
   - Capture du Dockerfile
   - Captures Nexus (SNAPSHOT et RELEASE)
   - Capture Docker Hub
   - Contenu de `inventory.ini`
   - Contenu de `playbook.yml`
4. **Captures depuis la VM :**
   - Résultat de `docker ps`

- Résultat de `curl localhost:8085/api/tasks`

Tout dépôt privé ou livrable incomplet ne sera pas corrigé.

Date de livraison : <u>Dimanche 28 Février à 23h59</u>